

SMOT-APP Production Readiness Audit

Data: 20 Maggio 2026 — **Aggiornato 20 Maggio 2026 ore 18:30**
Commit attuale: 04cfd0d (origin/main) **Metodo:** Analisi codice sorgente Rust (966 righe lib.rs), React (App.tsx, ErrorBoundary), Tauri config, CI/CD workflows **Team:** BigBoss133 (Michele), salvograsso10 (Salvatore), osurac5 (Tommaso)

Riepilogo

- **Layer analizzati:** 10
- **Layer pronti:** 3/10 → 4/10 (+1 Osservabilità base)
- **Layer parzialmente pronti:** 4/10 → 5/10 (+1 Affidabilità migliorata)
- **Layer non pronti:** 3/10 → 1/10
- **Gap critici (P0):** 8 → 5 (3 risolti da Michele)
- **Gap importanti (P1):** 12 → 7 (5 risolti da Michele)
- **Gap nice-to-have (P2):** 7
- **Stima effort residuo:** 3-4 settimane (Salvatore + Tommaso)
- **Progresso Michele:** 8/8 task P0+P1 completati 

Audit per Layer

1. Sicurezza — Parzialmente pronto

Check	Stato	Note
Input validation (path traversal)		<code>validate_upload_file_path()</code> robusto: <code>..</code> detection, reserved Windows names
Input validation (chat query)		<code>validate_chat_query()</code> : empty check, max 1000 chars

Check	Stato	Note
Input validation (document ID)	✓	<code>validate_document_id()</code> : UUID format validation via <code>uuid::Uuid::parse_str</code>
Input validation (model name)	✓	<code>pull_ollama_model</code> : alphanumeric + chars check
Rate limiting (Ollama)	✓	<code>check_rate_limit()</code> : 5 req/sec con finestra scorrevole
SQL injection prevention	✓	Prepared statements ovunque in <code>db.rs</code> e <code>indexing.rs</code>
CSP (Content Security Policy)	⚠	<code>'unsafe-inline'</code> per <code>style-src</code> — permette XSS via CSS injection
Secret management	✓	Nessuna chiave hardcoded nel codice Rust
Code signing macOS	✗	<code>"macOS": {}</code> vuoto — nessuna configurazione code signing
Code signing Windows	✗	Nessuna configurazione EV signing per Windows
Notarization macOS	✗	Nessuna configurazione notarization
Tauri permissions	⚠	Nessuna restrizione esplicita su <code>fs</code> scope (permisioni filesystem)
HTTPS enforcement	N/A	App locale, non usa HTTP pubblico

2. Affidabilità — ⚠ Parzialmente pronto

Check	Stato	Note
Error handling tipizzato	✓	<code>AppError</code> enum con 9 varianti + <code>From<></code> conversion + <code>Context</code> trait
Error per tutti i comandi Tauri	✓	Ogni <code>#[tauri::command]</code> restituisce <code>Result<T, String></code>
Graceful degradation (Ollama)	✓	Fallback message se Ollama non raggiungibile nella chat
ErrorBoundary React	✓	<code>ErrorBoundary</code> component wrappa l'intera app
Crash recovery	✗	

Check	Stato	Note
		Nessun panic handler custom in Rust. Crash = app chiusa senza recovery
Retry logic	✗	Nessun retry per operazioni fallibili (Ollama API, DB)
Health check endpoint	✗	Nessun comando Tauri per verificare lo stato interno
Circuit breaker	✗	Nessuna protezione contro failure a cascata
Graceful shutdown	⚠	<code>WindowEvent::CloseRequested</code> <code>prevent_close</code> + <code>hide</code> . OK per UX, ma nessuna cleanup DB/file

3. Performance — ⚠ Parzialmente pronto

Check	Stato	Note
Query optimization	✓	FTS5 con snippet, LIMIT 5, parametrizzate
Connection pooling	✗	Singolo <code>Mutex<Connection></code> — blocca tutte le operazioni concorrenti
Caching (Ollama)	✓	<code>OLLAMA_CACHE</code> con 5s TTL
Embedding in batch	✗	Embedding uno alla volta, nessun batching
Bundle size	✓	372 KB JS + 16 KB CSS — ottimo
Lazy loading	✗	Nessun <code>React.lazy()</code> o <code>Suspense</code> — tutte le pagine caricate subito
Async indexing	✓	<code>tauri::async_runtime::spawn</code> con progress events
Database vacuum/optimize	✗	Nessuna manutenzione automatica DB
Indexing parallelism	✗	Single-thread, documenti uno alla volta

4. Observability — ✗ Non pronto

Check	Stato	Note
Structured logging	✗	Solo <code>log::info!</code> e <code>log::warn!</code> con stringhe semplici. Nessun correlation ID, nessun livello warn/error strutturato
Metrics	✗	Nessuna metrica (tempo richieste, error rate, indexing throughput)

Check	Stato	Note
Crash reporting	✗	Nessun Sentry, nessun panic hook custom
Log rotation	⚠	Directory <code>logs/</code> creata ma nessun file di log visibile
Debug logging	⚠	<code>tauri_plugin_log</code> solo in <code>debug_assertions</code> — nessun log in produzione
Telemetry	✓	Nessuna telemetria (by design, app offline)
Performance tracing	✗	Nessuna instrumentation (tracing, OpenTelemetry)

5. Testing — ⚠ Parzialmente pronto

Check	Stato	Note
Rust unit tests	✓	62 test (lib.rs: 13, error.rs: 21, indexing.rs: 20, più <code>system_probe</code>)
Input validation tests	✓	Path traversal, reserved names, UUID, chat query, rate limits
E2E tests (Playwright)	⚠	Solo 3 test navigation. Nessun test per upload, chat, settings, indexing
React component tests	✗	Zero unit test React (nessun Vitest/Jest per componenti)
Integration tests	✗	Nessun test end-to-end del flusso completo (upload → indexing → chat)
Load testing	✗	Nessun test di carico (100+ documenti, query concorrenti)
Security scanning	✗	Nessun <code>cargo audit</code> , nessun <code>npm audit</code> nel CI
Cross-platform testing	✓	CI su Windows/macOS/Linux

6. CI/CD — ⚠ Parzialmente pronto

Check	Stato	Note
Build matrix (3 OS)	✓	Windows, macOS, Linux nel CI
Rust tests in CI	✓	<code>cargo test</code> eseguito nel CI

Check	Stato	Note
Rust lint in CI	✓	<code>cargo clippy</code> nel CI
Frontend lint in CI	✗	Nessun <code>npm run lint</code> o <code>tsc --noEmit</code> nel CI
Artifact publishing	✗	Build produce bundle ma non pubblica release draft
Version bump automation	✗	Versione manuale in <code>tauri.conf.json</code>
Dependency auditing	✗	Nessun <code>cargo audit</code> o <code>npm audit</code> automatico
Test coverage reporting	✗	Nessun code coverage tool
Pre-commit hooks	✗	Nessun husky/lint-staged

7. UX Production — ⚠ Parzialmente pronto

Check	Stato	Note
Loading states	✓	Skeletons in Dashboard, Chat, Viewer; indexing progress bar
Empty states	✓	Empty states per pagine vuote
Error UI	✓	ErrorBoundary con fallback UI
Toast notifications	✓	Success/error/info/warning toasts
Accessibility (a11y)	✗	Solo 5 attributi aria/role — insufficiente per WCAG AA
i18n	⚠	Italiano/English implementato, ma non tutte le stringhe sono tradotte
Dark mode	✗	Nessun tema scuro/chiaro
Responsive design	⚠	Finestra minima 1200x760 — non mobile-friendly
Keyboard navigation	✗	Nessun focus management, nessun tabIndex verificato
Offline mode UX	✗	App si dichiara offline ma non mostra stato connettività

8. Installer & Distribution — ⚠ Parzialmente pronto

Check	Stato	Note
NSIS installer (Win)	✓	Configurato con header/welcome images, lingue IT/EN

Check	Stato	Note
DMG bundle (Mac)	⚠️	Configurato ma senza code signing
Linux packages	✅	deb e AppImage configurati
Auto-update	⚠️	Tauri updater configurato con pubkey ma nessun test
Update rollback	❌	Nessuna strategia rollback se update fallisce
Installer signing	❌	Nessun code signing su Windows o macOS
Version management	⚠️	0.1.0-beta vs 0.1.0 conflitto (commit 5d294c7 fix per MSI)
Silent install	✅	installMode: "passive" per Windows updater

9. Data & Storage — ⚠️ Parzialmente pronto

Check	Stato	Note
Schema migrations	⚠️	Solo 001_initial.sql — nessun meccanismo di migrazione versionato
Backup	✅	backup_database e restore_database implementati (commit 04cfd0d)
Data export	❌	Nessuna funzionalità export documenti/indici
Data integrity	✅	Chiavi esterne, ON DELETE CASCADE, FTS5
File storage	✅	App data dir strutturata (documents/, thumbnails/, logs/)
Disk space check	✅	check_disk_space() per installazione modelli

10. Compliance — ❌ Non pronto

Check	Stato	Note
GDPR compliance	❌	Nessuna privacy policy, nessun consenso dati
Data retention policy	❌	Nessuna policy documentata
License terms	❌	Nessun EULA o termini di servizio
Open source licenses	⚠️	Dipendenze Rust/JS non attribuite
Accessibility compliance	❌	Non conforme WCAG 2.1 AA

Piano d'Azione Prioritizzato

P0 — Bloccanti per produzione (PRIMA di qualsiasi release pubblica)

#	Task	File/Riferimento	Effort	Assegnato
P0.1	Aggiungere panic handler e crash reporting	main.rs , nuovo file panic_handler.rs	2gg	Michele
P0.2	Implementare structured logging (correlation ID, livelli, file rotation)	lib.rs , setup.rs , nuovo crate tracing	2gg	Michele
P0.3	Fix CSP: rimuovere 'unsafe-inline' da style-src	tauri.conf.json , refactor stili in CSS file	1gg	Salvatore
P0.4	Aggiungere npm run lint e tsc --noEmit al CI	.github/workflows/build-*.yml	0.5gg	Tommaso
P0.5	Configurare code signing macOS (Apple Developer cert)	tauri.conf.json , CI secrets	2gg	Tommaso
P0.6	Fix doppio import in App.tsx (merge artifact)	smot-desktop/src/App.tsx	0.1gg	Salvatore
P0.7	Aggiungere unit test React (Vitest + Testing Library)	smot-desktop/src/__tests__ /	3gg	Salvatore
P0.8	Implementare database connection pooling (r2d2)	lib.rs , db.rs	1gg	Michele

P1 — Importanti (prima della beta pubblica)

#	Task	File/Riferimento	Effort	Assegnato
P1.1	Aggiungere E2E test per flussi critici (upload→index→chat)	smot-desktop/e2e/	3gg	Salvatore
P1.2		ollama.rs , indexing.rs	1gg	Michele

#	Task	File/Riferimento	Effort	Assegnato
	Aggiungere retry logic per Ollama API con exponential backoff			
P1.3	Implementare <code>React.lazy()</code> + <code>Suspense</code> per tutte le pagine	<code>App.tsx</code> , tutte le pagine	1gg	Salvatore
P1.4	Aggiungere <code>cargo audit</code> + <code>npm audit</code> al CI	<code>.github/workflows/build-*.yml</code>	0.5gg	Tommaso
P1.5	Implementare auto-update rollback strategy	<code>tauri.conf.json</code> , nuovo file <code>updater.rs</code>	1gg	Tommaso
P1.6	Aggiungere dark mode toggle	<code>src/context/ThemeContext.tsx</code> , <code>src/index.css</code>	1gg	Salvatore
P1.7	Rendere Ollama model configurabile (da <code>config.json</code>)	<code>ollama.rs</code> , <code>setup.rs</code>	0.5gg	Michele
P1.8	Aggiungere keyboard navigation e focus management	Tutti i componenti UI	2gg	Salvatore
P1.9	Implementare backup/restore database	<code>db.rs</code> , nuovo comando Tauri	2gg	Michele
P1.10	Migliorare accessibilità (WCAG AA): ARIA labels, focus, contrast	Tutti i componenti	2gg	Salvatore
P1.11	Aggiungere graceful shutdown (DB checkpoint, cleanup)	<code>lib.rs</code> <code>on_window_event</code>	0.5gg	Michele
P1.12	Aggiungere batching per embeddings Ollama	<code>indexing.rs</code>	1gg	Michele

P2 — Nice-to-have (post-launch)

#	Task	File/Riferimento	Effort	Assegnato
P2.1			1gg	Tommaso

#	Task	File/Riferimento	Effort	Assegnato
	Aggiungere code coverage reporting nel CI	<code>.github/workflows/</code> , <code>tarpaulin.toml</code>		
P2.2	Implementare schema migration versionato	<code>src-tauri/migrations/</code>	1gg	Michele
P2.3	Aggiungere GDPR privacy policy e consenso	Nuovo file <code>PRIVACY.md</code>	1gg	Tommaso
P2.4	Aggiungere EULA / Termini di Servizio	Nuovo file <code>TERMS.md</code>	0.5gg	Tommaso
P2.5	Performance tracing con <code>tracing</code> crate	<code>lib.rs</code> , tutti i moduli	2gg	Michele
P2.6	Ottimizzare indexing con elaborazione parallela documenti	<code>indexing.rs</code>	2gg	Michele
P2.7	Aggiungere export documenti e indici	Nuovo comando Tauri	1gg	Michele

Cosa NON fare ora

- **Non aggiungere features nuove** (collaborazione real-time, sync cloud, sharing) — prima stabilizza ciò che esiste
- **Non preoccuparti del mobile** — Tauri mobile è immaturo, focalizzati su desktop
- **Non rifare l'UI da zero** — il design attuale è funzionale. Raffina, non riscrive
- **Non migrare a Tauri v3** — v2.11 è stabile. Migra solo quando v3 è GA
- **Non aggiungere supporto per altri LLM** (OpenAI, Claude API) — Ollama è il focus
- **Non implementare un orchestratore di modelli** — un modello alla volta è sufficiente per v1

Stato Build Attuale

Comando	Risultato
<code>cargo build</code>	✓ 0 errori
<code>cargo test</code>	✓ 65/65 test passano
<code>cargo clippy -- -D warnings</code>	✓ Clean
<code>npx tsc --noEmit</code>	✓ 0 errori
<code>npx eslint 'src/**/*.{ts,tsx}'</code>	✓ 0 errori, 0 warning
<code>npm run build</code>	✓ 372 KB JS, 16 KB CSS
<code>npx playwright test</code>	✓ 3/3 E2E test passano

Verdict

L'app è in uno stato **alpha avanzato** → **beta iniziale**. I pilastri core funzionano (upload, indexing, ricerca, chat RAG). Progressi significativi di Michele:

- ✓ **Crash reporting:** panic hook scrive crash-report.json, rilevato al prossimo avvio
- ✓ **Structured logging:** `tracing` crate con livelli (DEBUG in dev, INFO in prod), file + linea
- ✓ **DB connection pooling:** r2d2 con pool da 4 connessioni, niente più Mutex bottleneck
- ✓ **Retry logic:** 3 tentativi con exponential backoff (1s, 2s, 4s) per embeddings Ollama
- ✓ **Model configurabile:** `ollama_model` in config.json, fallback a llama3.1:8b
- ✓ **Backup/restore DB:** `backup_database` e `restore_database` Tauri commands
- ✓ **Batching embeddings:** 3 richieste concorrenti per embeddings Ollama

Rimangono da fare i task di Salvatore (frontend: CSP, test React, lazy loading, dark mode, a11y, keyboard nav) e Tommaso (CI, code signing, audit deps, auto-update rollback, GDPR).

Con 3-4 settimane di lavoro focalizzato sui task P0 e P1 rimanenti, l'app può raggiungere uno stato **beta pubblicabile**.

Audit generato da Atlas (Sisyphus Orchestrator) — 20 Maggio 2026